KEEL (CEOPRO-AI) - AI Technical Brief

Prepared from the repository state at `main` commit `041f9f2` (October 2026). Every statement below cites the file it comes from. Test results and live checks are the ones **recorded in the repository's own documents**. They were not re-run while preparing this brief.

1. Project Overview

KEEL (the repository is named CEOPRO-AI) is a multi-tenant web application. It turns a business's own sales, stock and pricing data, plus public market information, into daily decisions: restocking, repricing, competitor moves, expected demand and promotions. It runs in English and Arabic (`README.md`). The stack is a FastAPI backend (`backend/src/ai/main.py`), a React + Vite frontend (`frontend/`) and Postgres with row-level security, Redis and MinIO/S3-compatible storage.

What AI does in the system. Most of the "intelligence" is deterministic or classical. Learned models are used in four places:

Capability	Technique	Status
Business assistant (Q&A over company data and documents)	Hybrid retrieval plus a hosted LLM, with claim-level citation checking	Implemented
Demand forecasting	XGBoost, chosen only when it beats statistical baselines in a backtest	Implemented
Review sentiment	Pretrained XLM-RoBERTa classifier, with a star-rating fallback	Implemented
Marketing copy and images	The same LLM client for copy; hosted or self-hosted diffusion for images	Implemented, with optional parts

Pricing recommendations, the Market Perception Index, cross-signal insights, entity extraction and comment themes are **rule-based**. They involve no model (see Section 2.6).

2. AI Components and Functionality

2.1 Business assistant (RAG + advisor) - `backend/src/ai/rag/`, `backend/src/ai/research/`

- **Purpose:** answers questions about the company's records and uploaded documents. Every claim in the answer cites its evidence.
- **Input:** `POST /rag/query` (`routers/rag.py`) with the question, conversation state, tenant and user.
- **Output:** `answer`, `answer_summary`, `answer_blocks` (claims with reference numbers), `references`, `calculations`, `evidence_status`, `research`, `follow_up_questions` and `answer_mode` (`advisor`, or a fallback such as `evidence_mode` or `unverifiable`). See `rag/README.md` section 3.
- **Workflow** (`rag/advisor.py` docstring):
  1. Rule-based intent and slot detection in English and Arabic (`understanding.py`). No LLM call.
  2. Evidence gathering, limited to what the member's role may see (`access.py`). Sources are live fact sections (`structured_context.py`), tool outputs (`orchestrator.py`, `business_tools.py`) and document passages.
  3. Optional public web research (`research/research.py`). It reads the source pages and enforces daily search budgets.
  4. Deterministic landed-cost calculation (`trade/landed_cost.py`).
  5. The LLM drafts claims as JSON that cite evidence IDs. `citations.py` checks each claim, with one repair round.
  6. If the provider is unconfigured, unpriced, rate-limited or down, the advisor returns an evidence-only answer instead of an error (`advisor.py` around lines 787 and 1021-1060).
- **Retrieval pipeline** (`rag/pipeline.py`):
  - Chunking (`chunking.py`).
  - BM25Plus keyword index (`bm25_index.py`).
  - FAISS flat inner-product index (`faiss_index.py`).
  - Reciprocal Rank Fusion (`hybrid_retrieval.py`).
  - Optional Cross-Encoder re-ranking (`reranking.py`, `RAG_RERANKER_ENABLED`).
  - Chunks and 384-dimensional vectors are stored in pgvector (`rag_document_chunks`).
  - Document ingestion handles text, PDF (`pdfplumber`) and DOCX (`python-docx`).
- **Embedding backends** (`rag/embeddings.py`, `RAG_EMBEDDING_BACKEND`):
  - `torch` (default): `paraphrase-multilingual-MiniLM-L12-v2`.
  - `onnx`: the same model on onnxruntime.
  - `static`: `static-similarity-mrl-multilingual-v1`, with no torch.
  - `none`: keyword search only.
- **Persistence:** conversations are stored in `conversation_sessions` and `conversation_messages` (`conversation_store.py`), exposed through `GET /rag/sessions...`.
- **Frontend:** `frontend/src/features/knowledge-base/` (`RagAssistantPage.jsx`, `useRagChat.js`).

2.2 LLM client - `backend/src/ai/rag/llm_client.py`

- **Purpose:** the only module that calls a language model. The assistant and marketing copy both use it.
- **Providers:**
  - Groq (default model `openai/gpt-oss-20b`).
  - Hugging Face Inference.
  - A local OpenAI-compatible server (`LOCAL_LLM_BASE_URL`, for example llama.cpp).
  - A paid OpenAI-compatible vendor (`PAID_LLM_*`).
- **Fallback:** `LLM_PROVIDERS` sets an ordered fallback chain. Without it, the single active provider is used.
- **Resilience:** retries with backoff, `Retry-After` handling, a circuit breaker for each provider and a single `LLMError` hierarchy.
- **Metering:** every call goes through `billing/credits.py`. A provider with no recorded price is refused (`COST_NOT_CONFIGURED`).

2.3 Demand forecasting - `backend/src/ai/forecasting/`

- **Purpose:** 30-day demand forecast for each product.
- **Input:** daily sales history and product/stock context (`data_access.py`).
- **Output:** `demand_forecasts` and `evidence_records` rows, plus `model_versions` when a model is trained (`evidence.py`). Stock advice is in `stock_advice.py`.
- **Workflow** (`pipeline.py`):
  1. Cold-start check (`cold_start.py`, default 30 days of history).
  2. Baselines: naive, seasonal-naive and moving average (`baselines.py`).
  3. If the data is sufficient, XGBoost on lag, rolling and calendar features (`model.py`, `features.py`) with expanding-window walk-forward validation.
  4. The model is used only if it beats the best baseline. Otherwise the baseline is served.
- **Triggers:**
  - `forecasting/refresh.py` runs a background refresh inside the API process after sales imports, recorded sales, page visits and `POST /forecasting/refresh`.
  - `POST /dashboard/products/{id}/forecast` forecasts one product on demand.
  - The Redis-stream consumer (`consumer.py`) exists but is **not started by any deployment** (documented in `routers/dashboard.py` around line 405 and in `refresh.py`).
- **Evaluation:** `scripts/evaluate_forecasting.py` and `backtesting.py`. The only stored backtest (`backend/reports/forecasting/backtest-demo/`) uses **deterministic synthetic demo data**, so its metrics say nothing about real-world accuracy.
- **Frontend:** `frontend/src/features/forecasting/`.
- **Trained model files are not persisted.** `model_versions` stores metadata only (`src/ai/README.md`, "Known upstream blockers").

2.4 Sentiment analysis - `backend/src/ai/sentiment/`

- **Purpose:** classify reviews and competitor comments as positive, neutral or negative, then aggregate by subject.
- **Model:** `cardiffnlp/twitter-xlm-roberta-base-sentiment` (`SENTIMENT_MODEL`), CPU, batched. torch and transformers are loaded lazily.
- **Workflow:**
  - `pipeline.classify_and_store_reviews()` writes `sentiment_results`.
  - When the model cannot load, it falls back to the review's star rating. Reviews without a rating stay unanalyzed.
  - `get_subject_sentiment_summary()` applies a LOW SAMPLE SIZE policy (`SENTIMENT_MIN_SAMPLE_SIZE`, default 10) and writes evidence.
- **Triggers:**
  - `POST /sentiment/analyze-pending` and `routers/reviews.py`.
  - The market analysis worker (`src/market_scraper/analysis_worker.py`). That worker runs in `docker-compose.yml` but is **disabled on staging** in `render.yaml` because it needs a paid plan.
- **Consumers:** the assistant (`structured_context.py`, `business_analysis.py`), MPI and insights.
- **Not connected:** `finetune.py` (a fine-tuning script) exists, but nothing in the application calls it.

2.5 Marketing AI - `backend/src/ai/marketing/`, `backend/image_server/`

- **Copy:** `content_generator.py` uses `llm_client` with a marketing prompt that returns structured JSON. Audience text is labelled as an AI suggestion. There is no vision input and no performance tracking (module docstring).
- **Images:** `image_generation.py` tries providers in order and crops each result to one of five formats. If every provider fails, `designed_poster.py` renders a non-AI poster.
  - Providers: local server, Cloudflare Workers AI, Hugging Face, Gemini, Together, AI Horde and Pollinations.
  - Order is set by `MARKETING_IMAGE_PROVIDERS`.
  - Images are kept per campaign in `campaign_image_assets`.
- **Self-hosted images:**
  - `backend/image_server/server.py` serves SDXL 1.0 base through `diffusers`, behind an OpenAI-style images API.
  - `image_jobs.py` and `image_worker.py` form a background queue with retries, crash recovery and fair turns between companies.
  - Both run under the Compose profile `local-images`.
- **Optional or not built:**
  - Background removal (`background_removal.py`, rembg) is opt-in and not installed by default (`requirements-rembg.txt`).
  - Video generation (`video_generation.py`) was investigated and **not implemented**.
- **API and frontend:** `/marketing/...` endpoints (`routers/marketing.py`) and `frontend/src/features/marketing/`.

2.6 Rule-based analytics the AI layer depends on

Module	What it does
`pricing/`	Same-currency market-average recommendation with a 15% guardrail. Not a learned model.
`mpi/scoring.py`	Market Perception Index (0-100) built from sentiment, source reliability, recency, volume and relevance
`insights/`	Rule engine over pricing, sentiment, sales trend and forecast signals
`extraction/`	Regex and catalog-matching NER, plus the CSV/XLSX/PDF/DB/API import engine
`market_inputs/comment_themes.py`	Keyword themes in English and Arabic

3. Technologies and Dependencies

Used in the code paths described above:

Technology	Role	Where
Groq API (`openai/gpt-oss-20b`), through `httpx`	Default LLM for the assistant and copy	`rag/llm_client.py`, `render.yaml`
OpenAI-compatible local server (llama.cpp; Qwen models in docs)	Optional LLM fallback	`llm_client.py`, Compose service `llm-local`
sentence-transformers, onnxruntime, static embedding model	Document and question embeddings	`rag/embeddings.py`
faiss-cpu, rank-bm25	Semantic and keyword retrieval	`rag/faiss_index.py`, `rag/bm25_index.py`
Cross-Encoder `mmarco-mMiniLMv2-L12-H384-v1`	Re-ranking (can be turned off)	`rag/reranking.py`
Postgres + pgvector	Chunk and vector storage, plus all AI outputs	migrations under `src/infrastructure/database/`
transformers + torch (XLM-RoBERTa)	Sentiment classification	`sentiment/model.py`
xgboost (scikit-learn needed internally), pandas, numpy	Demand forecasting	`forecasting/`
pdfplumber, python-docx, openpyxl	Document and file ingestion	`rag/data_access.py`, `extraction/adapters/`
Web search: Google CSE, Brave, Tavily or SearXNG	Public research for the assistant	`src/market_scraper/web_search.py`, `research/`
Hosted image APIs, Pillow	Campaign images and poster fallback	`marketing/image_generation.py`, `designed_poster.py`
diffusers, torch, SDXL 1.0 base	Self-hosted image server	`backend/image_server/` (separate requirements file)
Redis	Locks, caches, circuit-breaker and cooldown state, worker streams	`core/redis_client.py`, `forecasting/refresh.py`

Installed but limited or not in the request path:

- `accelerate`: needed only by the fine-tuning script `sentiment/finetune.py`, which nothing calls.
- `rembg`: in a separate opt-in requirements file, off by default.
- `pytesseract`: optional OCR; it reports "skipped" when the binary is missing.
- `scikit-learn`: pinned only because XGBoost needs it (`requirements.txt` comment).

4. Architecture and Integration

```
Frontend (React)
  knowledge-base/  forecasting/  marketing/  market-intelligence/
        |  HTTPS + JWT (tenant + role)
        v
FastAPI  backend/src/ai/main.py   (RLS connection per request; plan entitlements; rate limits)
  /rag/query -----> advisor.py
                      understanding (rules) -> access-checked evidence:
                        structured_context (sales, forecasts, sentiment, prices)
                        pipeline.run_retrieval (BM25 + FAISS -> RRF -> re-rank) <- pgvector chunks
                        research/ (web search provider -> page reader)       [optional]
                        trade/landed_cost (deterministic)
                      -> llm_client (Groq | local | HF | paid) -> citations check
                      -> evidence-only answer if the LLM is unavailable or unpriced
  /forecasting/*, /dashboard/products/{id}/forecast -> forecasting/refresh -> pipeline (baselines | XGBoost)
  /sentiment/*, /reviews/* -> sentiment/pipeline (XLM-R | star-rating fallback)
  /marketing/*  -> content_generator -> llm_client
                -> image_generation (local | hosted chain) | designed_poster | image_jobs queue
                                                                  -> image_worker -> image_server (SDXL)
  billing/credits.py meters every paid model and API call (refuses when no price is recorded)

Workers (docker-compose; disabled on Render staging):
  market_scraper/analysis_worker -> sentiment + MPI scores + structured RAG summaries
Storage: Postgres (RLS) | MinIO/S3 (documents, generated images) | Redis
```

Integration points:

- **Outputs written to the database.** Forecasts, sentiment, MPI and price comparisons are stored in Postgres. The assistant reads them from there; it does not call the modules directly.
- **Generated summaries.** `rag/structured_summaries.py` turns analytics results into retrievable text chunks.

- **Billing and entitlements.** Each AI router is mounted with plan entitlements in `main.py` (`requires(usage.DEMAND_PREDICTION)` and similar).
- **Tenant isolation.** Every query runs on a Postgres row-level-security connection (`db.app_role_connection`).

5. Current Status and Next Steps

Implemented and verified (according to the repository's records)

- **Test suites:** `docs/AI_MODEL_MARKETING_IMPLEMENTATION.md` §7.2 (2026-10-22) records 3,588 backend tests passed, 23 skipped and 0 failed, plus 113 frontend unit tests passed. `CURRENT_STATUS.md` holds an older run.
- **CI on real retrieval models:** `.github/workflows/rag-runtime-check.yml` (2026-10-17) confirmed that the embedding and re-ranking models load, that an Arabic question retrieves an English document, and that the static backend peaks at 359 MB.
- **Live assistant check:** recorded on 2026-10-09 with `scripts/live_llm_verification.py` (`CURRENT_STATUS.md`).
- **Real SDXL images:** 17 images went through the full API, queue, worker, server and gallery path. Measured throughput was about 24-27 images per hour on a 4-core CPU, with a 10.2 GB memory peak (`docs/ZERO_COST_IMAGE_GENERATION.md`). Copy written by a local Qwen3-4B model also worked through the fallback chain.

Incomplete, unverified or blocked

- **Staging API not answering.** On 2026-10-21 it gave no response on `/health` (`docs/FINAL_SYSTEM_AUDIT.md` §3.2). The root cause is not confirmed; missing required secrets are the leading candidate.
- **Assistant cannot write answers on hosted environments yet.** It needs a `GROQ_API_KEY` **and** a recorded Groq price (`backend/PENDING_ACTIONS.md` #124, #125). Until both exist, answers fall back to evidence mode.
- **Exposed credentials.** Credentials were committed to public Git history and must be rotated (`FINAL_SYSTEM_AUDIT.md` §3.1, `docs/SECURITY_CREDENTIAL_ROTATION.md`). The configured Hugging Face token has expired (PENDING #145).
- **Not checked live from the development environment:** model downloads from Hugging Face, Google, Apify, Stripe and real object storage (`CURRENT_STATUS.md`, "Known limits").
- **Answer quality:**
  - Counting questions ("How many products do I have?") are often answered with "not enough information" (PENDING #130).
  - Follow-up questions that don't name the product sometimes miss the earlier product (1 of 3 live runs).
  - `ADVISOR_MAX_TOKENS` has not been tuned against the real model (#137).
- **No answer-accuracy evaluation exists.**
  - `rag/eval/run_eval.py` checks gate selection and grounding mechanics; it does not judge answer correctness.
  - There is no side-by-side comparison of the local and Groq models (`rag/README.md`).
  - Forecast accuracy has only been measured on synthetic demo data.
- **Static embeddings on the 512 MB staging plan** are described as less accurate than the transformer models (`CURRENT_STATUS.md`).
- **Free Groq tier limits.** Under load, replies wait 9-16 seconds, and the 200,000-token daily cap was exhausted during QA (PENDING #135).
- **Unused code paths:**
  - The forecasting Redis consumer is never started.
  - `sentiment/finetune.py` is never called.
  - Trained forecast model files are not stored.
  - Video generation is not built.
- **Outdated documentation.** `backend/src/ai/README.md` says PDF/DOCX ingestion and chat history are missing, but both now exist in code. The README itself says it predates later rework.

Questions for the meeting

1. **Staging recovery:** who has Render access to read the logs, restore the API and enter `GROQ_API_KEY`, the Groq price and storage credentials?
2. **LLM provider policy:** should `LLM_PROVIDERS` be set on staging/production, and which providers may receive company passages (#147)? Also confirm the licence of the default local model, Qwen2.5-3B-Instruct (#144, believed non-commercial).
3. **Hosting size for models:** stay on the 512 MB plan (static embeddings, no re-ranker, workers off), or move to 2 GB or more for ONNX embeddings and the sentiment worker? It is not documented whether the XLM-R sentiment model can load on the 512 MB service; the star-rating fallback covers rated reviews only.
4. **Evaluation:** agree a labelled question set and an accuracy measure for the assistant, and a backtest on real (or realistic seeded) sales for forecasting, before quality claims are made.
5. **Public research provider:** which web search provider to use in production (#131)? The assistant's tax, customs and market questions depend on it.
6. **Cleanup:** wire or remove the unused forecasting consumer and `finetune.py`; decide whether to persist trained forecast models; update `backend/src/ai/README.md`.